# Evolution Processes (عمليات التطوير)

## المقدمة — لماذا يتغير البرنامج؟

- الأنظمة الكبيرة لها عمر طويل جداً — أنظمة عسكرية تعيش **30 سنة أو أكثر**، وأنظمة الأعمال تتجاوز **10 سنوات** عادةً.

**أرقام مهمة لا تنساها:**

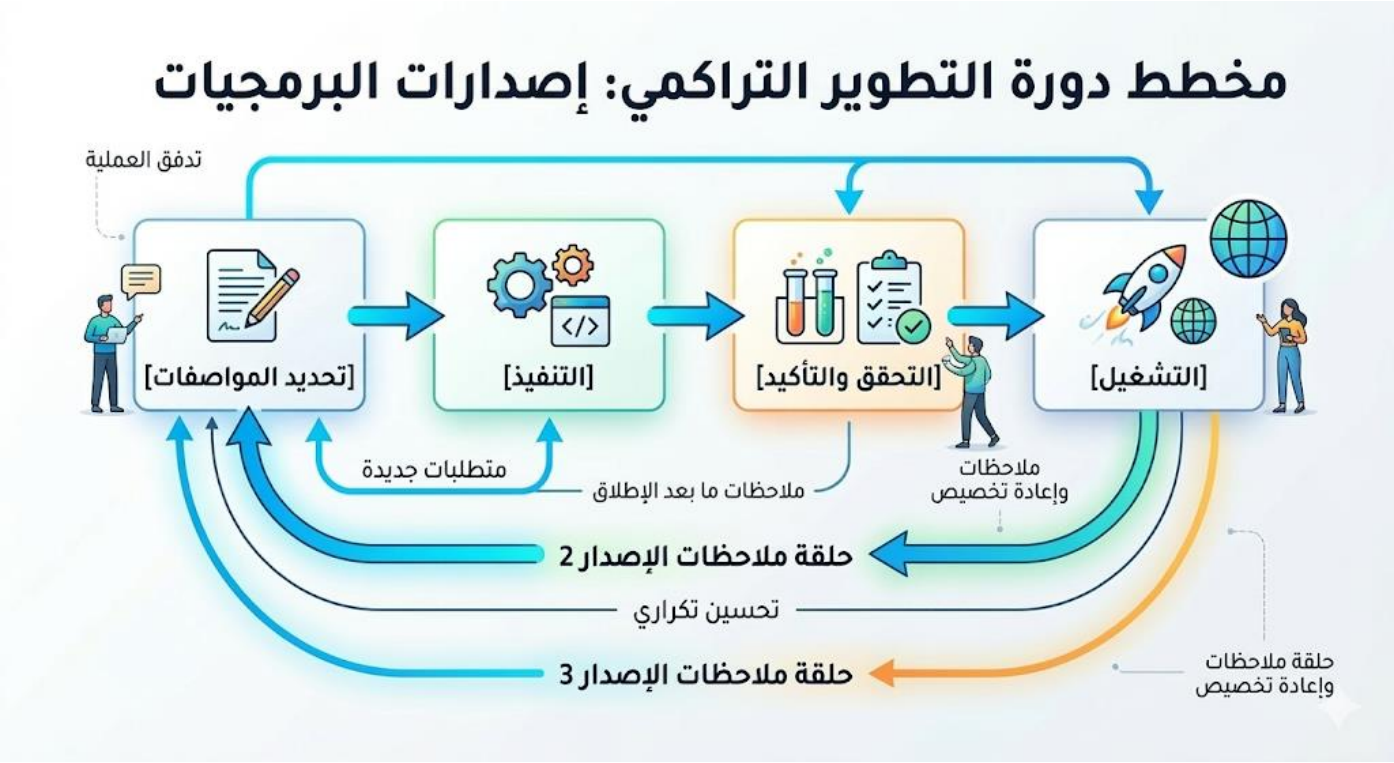
- بين **60%** و**90%** من تكاليف البرمجيات = تكاليف Evolution.
- **75%** من موظفي التطوير في أمريكا (2006) يعملون في Software Evolution.

**الفكرة الأساسية:** هندسة البرمجيات ليست مجرد بناء نظام مرة واحدة — هي **عملية حلزونية مستمرة** طوال حياة النظام.

---

## نموذج الحلزون

البرمجيات تتطور على شكل **Spiral** — كل إصدار يبدأ فوراً بعد تسليم الإصدار السابق:




**قبل الإنترنت:** إصدار جديد كل **2-3 سنوات** — **الآن:** إصدار كل **أسابيع** بسبب المنافسة الشديدة!

---

## 🔄 نموذج Bennett & Rajlich — مراحل حياة البرنامج

يُميّز بين **4 مراحل** متتابعة:

Software Development
↓
Software Evolution ← تغييرات كبيرة في البنية والوظائف
↓ (تتداخل)
Software Servicing ← تغييرات صغيرة وضرورية فقط
↓
Software Retirement ← التقاعد والاستبدال

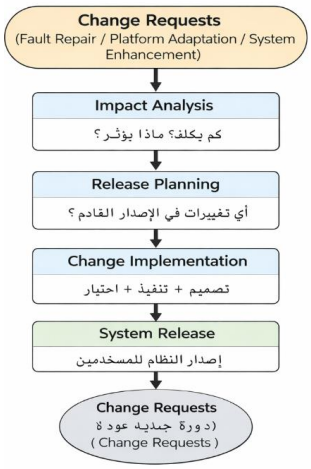
| المرحلة | ما يحدث فيها |
|---|---|
| **Evolution** | تغييرات جوهرية في المعمارية والوظائف — يقترحها أصحاب المصلحة. |
| **Servicing** | تغييرات تكتيكية صغيرة — الشركة تبحث عن البديل. |
| **Retirement** | استخدام محدود — المستخدم يتحايل على المشاكل حتى الاستبدال. |

⚠ **نقطة الانتقال:** بعد سنوات من التعديلات، تتدهور البنية وتصبح التغييرات الكبيرة **غير مجدية مادياً** ← تنتقل من Evolution إلى Servicing.

---

## ⚙ النموذج العام لعملية التطوير — Figure 9.4

مُحرّك التطوير = **Change Proposals** (مقترحات التغيير):




**مصادر مقترحات التغيير (Change Proposals):**

- متطلبات موجودة **لم تُنفَّذ** في الإصدار الحالي.
- طلبات متطلبات جديدة.
- تقارير أخطاء من أصحاب المصلحة.
- أفكار تحسين من فريق التطوير.

---

## Program Understanding — الفرق الحاسم

عندما يتعامل مع التغيير **فريق مختلف** عن فريق التطوير الأصلي، تبرز مرحلة إضافية هي:

**Program Understanding** = الفريق الجديد يجب أن يفهم:

1. كيف يتكون هيكل البرنامج.
2. كيف يُقدّم وظائفه.
3. كيف سيؤثر التغيير المقترح على بقية النظام.

بدون هذا الفهم قد يُسبّب التغيير الجديد **مشاكل غير متوقعة** في الأجزاء الأخرى.

---

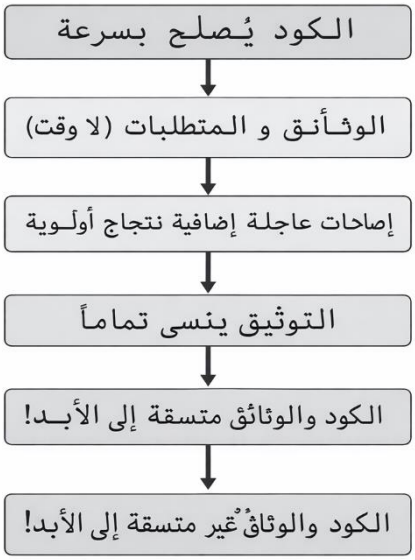
## Emergency Repair — الإصلاح العاجل

**Emergency Repair** = تغيير عاجل يتخطى الإجراءات الرسمية لإصلاح مشكلة حرجة **بأسرع وقت**.

**3 أسباب التغيير العاجل:**

| السبب | المثال |
|---|---|
| **خطأ حرج** يوقف التشغيل أو **ثغرة أمنية** | النظام يتوقف في المستشفى |
| **تغيير مفاجئ في بيئة التشغيل** يُعطل العمل | ترقية OS تُسبّب مشاكل غير متوقعة |
| **تغييرات غير متوقعة في الأعمال** | منافس جديد أو تشريع جديد |

**خطر Emergency Repair:**




هذا المشكلة هو أحد **أقوى مبررات** مبدأ **Minimal Documentation** في Agile.

**الحل المثالي:** إعادة **Refactoring** للكود الجديد لمنع تدهور البرنامج — لكنه نادراً ما يحدث.

---

## نوعان من Evolution — مهم للمقارنة!

| المعيار | Integrated Evolution (منتجات) | Custom Software Evolution |
|---|---|---|
| **من يطور؟** | نفس الشركة طوال الحياة | شركة تطوير ← شركة صيانة مختلفة |
| **الانتقال** | **سلس** — نفس الأساليب والفريق | **غير سلس** — انقطاع في العمليات |
| **الوثائق** | تنتقل بسلاسة | قد لا تنتقل بين الشركتين |
| **المثال** | تطبيقات ومنتجات Software | أنظمة مخصصة للحكومات والمستشفيات |
| **النتيجة** | يُسمى **Software Evolution** | يُسمى **Software Maintenance** |

---

## Agile في Evolution

**مميزات استخدام Agile في Evolution:**

- الانتقال من تطوير لـ Evolution **سلس** بطبيعة Agile التدريجية.
- **TDD + Automated Regression Testing** مفيدان جداً في التغييرات.
- **User Stories** لتمثيل طلبات التغيير.
- **Scrum Backlog** لترتيب أولويات التغييرات.

**مشكلتان عند تسليم النظام بين فريقين:**

1. **Agile → Plan-Based:** فريق Agile طوّر بدون وثائق مفصّلة → فريق Plan-Based يحتاجها.
2. **Plan-Based → Agile:** الكود القديم لم يُعاد هيكلته → يحتاج **Reengineering** قبل استخدام Agile.

**تعديلات Agile المطلوبة أثناء Evolution:**

| Maintenance / Evolution | التطوير | الجانب |
|---|---|---|
| صعب — طلبات التغيير من مصادر متعددة | مستخدم واحد في الفريق | المستخدم |
| Emergency Repairs قد تُقاطَع بـ | دورات قصيرة منتظمة | الدورات |
| قد تُطوَّل لتجنب تعطيل العمليات الجارية | إصدارات متكررة | الإصدارات |

---

## مصطلح مهم — Brownfield Development

**Brownfield Software Development** = تطوير نظام في بيئة **تعتمد على أنظمة أخرى موجودة** → يجب مراعاة تأثير التغيير على تلك الأنظمة المحيطة.

**المصدر:** Jenkins & Hopkins (2008) — عكسه **Greenfield** = بناء نظام من الصفر بدون قيود.

---

## نقاط MCQ احفظها جيداً

- ◈ **60-90%** من تكاليف البرمجيات = تكاليف Evolution — ليس التطوير الأولي!
- ◈ **75%** من موظفي التطوير يعملون في Software Evolution (Jones 2006).
- ◈ **Software Engineering = Spiral Process** — لا ينتهي بالتسليم.
- ◈ **Bennett & Rajlich** = 4 مراحل: Development → Evolution → Servicing → Retirement.
- ◈ **Evolution** = تغييرات كبيرة في البنية | **Servicing** = تغييرات صغيرة فقط.
- ◈ **مُحرّك التطوير = Change Proposals** من مصادر متعددة.
- ◈ **4 أنشطة أساسية:** Impact Analysis → Release Planning → Change Implementation → System Release.
- ◈ **Program Understanding** = مرحلة إضافية عند وجود فريق مختلف.
- ◈ **3 أسباب لـ Emergency Repair:** خطأ حرج / تغيير بيئة / تغيير أعمال.
- ◈ **خطر Emergency Repair** = **عدم اتساق** الكود مع الوثائق.
- ◈ **Minimal Documentation** في Agile = حل لمشكلة Emergency Repair.
- ◈ **Custom Software** غير سلس الانتقال → يُسمى **Maintenance** لا Evolution.
- ◈ **Brownfield** = بيئة تعتمد على أنظمة أخرى (Jenkins 2008 & Hopkins).
- ◈ **مشكلة Agile → Plan-Based** = غياب الوثائق المفصّلة.

---

# الأنظمة القديمة (Legacy Systems)

## تعريف Legacy Systems

**Legacy System** = نظام قديم يعتمد على لغات وتقنيات لم تعد مستخدمة في التطوير الجديد — صُوِّن على مدى سنوات طويلة وتدهورت بنيته بسبب التعديلات المتراكمة.

**النقطة الأساسية:** Legacy System ليس مجرد برنامج — هو نظام اجتماعي-تقني (**Sociotechnical System**) يشمل الأجهزة والبرامج والبرامج والبيانات وعمليات الأعمال!

**مثال حقيقي:** هناك أكثر من 200 مليار سطر **COBOL** لا تزال تعمل في الأنظمة المصرفية والمالية حتى اليوم — تعمل بكفاءة لكن المشكلة = شُح المبرمجين!

## الـ 6 مكونات لـ Legacy System — Figure 9.7

| المكوّن | تفصيل المشكلة |
|---|---|
| **System Hardware** | أجهزة غير متوفرة أو مكلفة الصيانة أو غير متوافقة مع سياسة IT الحديثة |
| **Support Software** | نظام تشغيل + مترجمات قديمة لم يعد موردوها يدعمونها |
| **Application Software** | برامج طُوِّرت في فترات مختلفة من قِبل فرق مختلفة — قد تكون جزءاً من أنظمة أخرى أيضاً |
| **Application Data** | بيانات ضخمة متراكمة — متكررة + غير متسقة + موزعة على قواعد بيانات متعددة |
| **Business Processes** | العمليات صُمِّمت حول النظام القديم ومقيّدة بإمكانياته |
| **Rules & Business Policies** | قواعد الأعمال مضمّنة في الكود — لم تُوثَّق في مكان آخر |

## طبقات Legacy System — Figure 9.8

Business Processes (الأعلى)
↑ تعتمد على
Application Software
↑ تعتمد على
Platform & Infrastructure Software
↑ تعتمد على
Hardware (الأسفل)

⚠ **المشكلة:** تغيير طبقة واحدة يؤثر على الطبقات الأعلى والأسفل — ليس مجرد الطبقة المجاورة — بسبب 3 أسباب:

1. الطبقات الأعلى قد تستغل مزايا التغيير الجديد.
2. تغيير البرنامج قد يُبطئ النظام فيحتاج أجهزة جديدة.
3. استحالة الحفاظ على واجهات الأجهزة عند التحديث.

---

## لماذا لا تستبدل الشركات Legacy Systems؟

السبب الجوهري: **استبدالها مكلف ومحفوف بالمخاطر.**

**4 أسباب تجعل الاستبدال خطيراً:**

| السبب | التفصيل |
|---|---|
| **غياب المواصفات الكاملة** | المواصفة الأصلية ضاعت أو لم تُحدَّث — لا يمكن بناء نظام مطابق |
| **تشابك عمليات الأعمال مع النظام** | العمليات تطورت لاستغلال النظام والتحايل على عيوبه — تغييرها بتكاليف غير متوقعة |
| **قواعد أعمال مضمّنة في الكود** | مثل: شركة تأمين تضع قواعد تقييم المخاطر في الكود — فقدانها = خسائر ضخمة |
| **خطر التطوير الجديد** | قد لا يُسلَّم في الوقت والتكلفة المتوقعين |

**مثال حقيقي من الكتاب:** شركة كبيرة قررت استبدال 150 نظاماً قديماً بنظام ERP مركزي واحد ← أنفقت 10 مليون جنيه إسترليني ← فشل المشروع ← النظام الجديد لم يعمل بكفاءة القديم ← اضطر المستخدمون للعمل بالنظامين معاً بمعالجة يدوية إضافية!

---

## 6 أسباب لارتفاع تكلفة تعديل Legacy Systems

1. أسلوب الكود غير متسق — كتبه أشخاص مختلفون على مر السنين
2. لغات برمجة متقادمة — صعب إيجاد مبرمجين لها
3. توثيق غير كافٍ أو قديم — أحياناً الكود هو الوثيق الوحيد
4. سنوات من الصيانة دمّرت البنية — أجزاء أضيفت بطريقة عشوائية
5. تحسينات للسرعة والمساحة جعلت الكود صعب الفهم
6. بيانات في ملفات غير متوافقة — تكرار وأخطاء وبيانات قديمة

---

# Legacy System Management (إدارة الأنظمة القديمة)

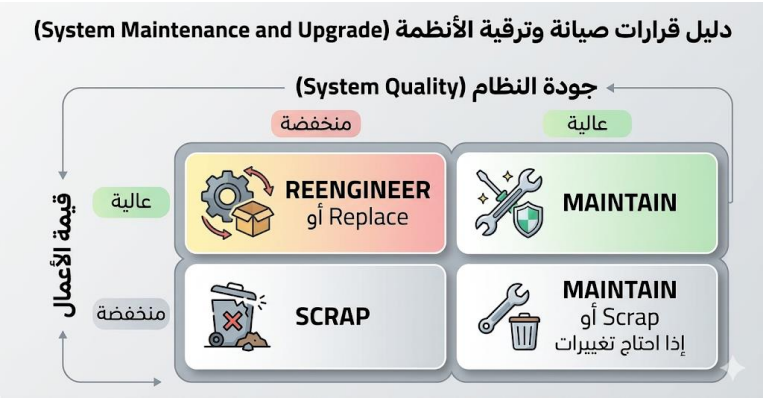
**الـ 4 استراتيجيات الرئيسية:**

| الاستراتيجية | متى تُختار؟ |
|---|---|
| ✕ **Scrap** (تخلص منه) | النظام لا يخدم العمليات فعلياً — عمليات الأعمال تغيرت ولم تعد تعتمد عليه |
| ☑ **Continue Maintenance** (استمر في صيانته) | النظام مطلوب + مستقر + طلبات التغيير قليلة |
| 🔧 **Reengineer** (أعد هندسته) | تدهورت الجودة + لا تزال هناك تغييرات مطلوبة — يمكن إضافة واجهات للتوافق مع الأنظمة الجديدة |
| 🔄 **Replace** (استبدله) | الأجهزة لا تستطيع الاستمرار أو أنظمة جاهزة (Off-the-shelf) متاحة بتكلفة معقولة |

---

## 🗺 مصفوفة القرار — Business Value × System Quality

التقييم يجب أن يشمل منظورين: **تجاري (Business) + تقني (Technical)**.




**تقييم 4 محاور Business Value —:**

| المحور | الأسئلة المطروحة |
|---|---|
| **استخدام النظام** | هل يُستخدم بكثرة؟ من يستخدمه؟ ⚠ الاستخدام النادر لا يعني انخفاض القيمة (مثال: نظام تسجيل الطلاب!) |
| **العمليات التجارية المدعومة** | هل النظام يفرض عمليات غير فعّالة؟ هل يمنع التطوير؟ |
| **موثوقية النظام** | هل الأعطال تؤثر مباشرة على العملاء؟ هل تستنزف موارد الشركة؟ |
| **مخرجات النظام** | هل الأعمال تعتمد على هذه المخرجات؟ يمكن الحصول عليها بطريقة أخرى؟ |

---

## تقييم System Quality — مؤشران

### أ) تقييم البيئة (Environment Assessment)

| العامل | السؤال |
|---|---|
| **Supplier Stability** | هل المورد لا يزال موجوداً وقادراً على الدعم؟ |
| **Failure Rate** | هل الأجهزة تتعطل كثيراً؟ هل البرامج المساعدة تتوقف؟ |
| **Age** | ما عمر الأجهزة والبرمجيات؟ |
| **Performance** | هل الأداء كافٍ؟ هل المشاكل تؤثر على المستخدمين؟ |
| **Support Requirements** | ما تكلفة الدعم المحلي؟ |
| **Maintenance Costs** | ما تكلفة صيانة الأجهزة وتراخيص البرامج؟ |
| **Interoperability** | هل يصعب ربط النظام بالأنظمة الأخرى؟ |

### ب) تقييم التطبيق (Application Assessment)

| العامل | السؤال |
|---|---|
| **Understandability** | هل الكود صعب الفهم؟ هل المتغيرات لها أسماء واضحة؟ |
| **Documentation** | هل التوثيق متاح ومحدّث ومتسق؟ |
| **Data** | هل هناك نموذج بيانات واضح؟ هل البيانات متكررة؟ |
| **Performance** | هل أداء التطبيق كافٍ؟ |
| **Programming Language** | هل تتوفر مترجمات حديثة للغة المستخدمة؟ |
| **Configuration Management** | هل إصدارات المكونات مُدارة بشكل صحيح؟ |
| **Test Data** | هل بيانات الاختبار ونتائج Regression Tests محفوظة؟ |
| **Personnel Skills** | هل يوجد موظفون لديهم خبرة بالنظام وبلغته؟ |

**ج) مؤشرات قياسية لتقييم الجودة:**

1. عدد طلبات التغيير — كلما ارتفع = جودة أقل.
2. عدد واجهات المستخدم — كثرتها = تضارب وتكرار.
3. حجم البيانات — بياناتان أكثر = أخطاء وعدم اتساق أكثر.

⚠ **ملاحظة واقعية مهمة:** أحياناً القرار ليس موضوعياً — قد يُبقي مدير على نظام رديء بسبب سياسات المؤسسة أو الدمج بين الشركات أو محدودية الميزانية في سنة معينة.

## نقاط MCQ احفظها جيداً

- ◈ **Legacy System** = نظام قديم يعتمد لغات/تقنيات لم تعد مستخدمة.
- ◈ هو **Sociotechnical System** — ليس مجرد برنامج.
- ◈ يتكون من **6 مكونات**: Hardware + Support SW + App SW + App Data + Business Processes + Policies/Rules.
- ◈ **200 مليار سطر COBOL** لا تزال في الأنظمة المصرفية.
- ◈ سبب الإبقاء عليه = الاستبدال أغلى وأخطر في معظم الحالات.
- ◈ **4 أسباب خطورة الاستبدال**: غياب المواصفات + تشابك العمليات + قواعد أعمال مضمّنة + خطر التطوير.
- ◈ تقييم Legacy = **Business Value + System Quality** معاً (Warren 1998).
- ◈ High business value + Low quality → **Reengineer**.
- ◈ Low business value + Low quality → **Scrap**.
- ◈ High business value + High quality → **Continue Maintenance**.
- ◈ Low business value + High quality → **Continue Maintenance** (أو Scrap عند الحاجة لتغييرات مكلفة).
- ◈ **4 محاور Business Value**: الاستخدام + العمليات المدعومة + الموثوقية + المخرجات.
- ◈ **7 عوامل Environment Assessment** تشمل: Supplier Stability + Failure Rate + Age + Performance + Interoperability ...
- ◈ الاستخدام النادر لا يعني انخفاض القيمة! (مثال: نظام تسجيل الطلاب).
- ◈ القرار أحياناً سياسي وليس موضوعياً بالكامل.

---

# Software Maintenance (صيانة البرمجيات)

## التعريف

**Software Maintenance** = العملية العامة لتغيير النظام بعد تسليمه — تُطبَّق عادةً على البرامج المخصصة (Custom Software) حيث يتولى فريق مختلف الصيانة بعد التسليم.

☆ **الحقيقة المهمة:** صيانة نظام موجود أغلى من تطوير نفس الميزة من الصفر — وهذا ما يُميّز الصيانة عن التطوير الأولي!

---

## 3 أنواع Software Maintenance — الأهم للامتحان

| النوع | الوصف | التكلفة النسبية |
|---|---|---|
| **Fault Repair (Bug Fixing)** | إصلاح أخطاء وثغرات أمنية | 24% من جهد الصيانة |
| **Environmental Adaptation** | تكييف البرنامج مع بيئات/منصات جديدة | 19% من جهد الصيانة |
| **Functionality Addition** | إضافة متطلبات جديدة | 58% من جهد الصيانة |

### تفصيل أنواع الأخطاء في Fault Repair:

- **Coding Errors** → رخيصة نسبياً.
- **Design Errors** → أكثر تكلفة (تحتاج إعادة كتابة مكونات).
- **Requirements Errors** → الأغلى (تحتاج إعادة تصميم واسعة).

**ملاحظة للامتحان:** كلمات بديلة شائعة في بعض الكتب:

- **Corrective Maintenance** = Fault Repair.
- **Adaptive Maintenance** = Environmental Adaptation أو إضافة متطلبات جديدة (الكلمة غامضة!).
- **Perfective Maintenance** = Functionality Addition أو تحسين البنية.
- الكتاب يتجنب هذه التسميات بسبب الغموض ويستخدم التسميات الأوضح.

---

## قوانين Lehman — Program Evolution Dynamics

Les Belady (1970s) & Manny Lehman — وضعا "Lehman's Laws" وهي القوانين التي تنطبق على جميع الأنظمة الكبيرة:

1. البرنامج يجب أن يتغير باستمرار ليبقى مفيداً
2. كلما تغيّر النظام، تتدهور بنيته
3. معدل التغيير ثابت تقريباً بغض النظر عن الموارد المتاحة
4. التغيير الإضافي في كل إصدار ثابت تقريباً
5. يجب إضافة وظائف جديدة لزيادة رضا المستخدمين

---

## لماذا الصيانة أغلى من التطوير؟ — 4 أسباب

| السبب | التفصيل |
|---|---|
| **الفريق الجديد يحتاج للفهم أولاً** | الفريق الأصلي انتقل لمشاريع جديدة — الفريق الجديد يصرف وقتاً طويلاً لفهم النظام قبل التعديل |
| **لا حافز لكتابة كود قابل للصيانة** | عقد الصيانة منفصل عن عقد التطوير — الفريق المطوّر لن يستفيد من جعل الكود سهل الصيانة |
| **الصيانة غير محبوبة** | تُعطى للمطورين الأقل خبرة — لغات قديمة صعبة — تُعدّ نشاطاً من الدرجة الثانية |
| **تدهور البنية مع الزمن** | كل تعديل يزيد التعقيد — الكود الأصلي ربما لم يكن مهيكلاً أصلاً — التوثيق قد يُفقد |

**الحل الجذري** = دمج التطوير والصيانة في فريق واحد مسؤول طوال حياة النظام — مثل Amazon التي تطوّر وتصون برامجها بنفسها.

**لماذا الشركات لا تستثمر في Maintainability؟**

- المدراء يُحفَّزون على تقليل التكاليف قصيرة المدى.
- المطورون لن يستفيدوا من صيانة النظام لاحقاً — فلماذا يبذلون جهداً إضافياً؟

---

# Maintenance Prediction (التنبؤ بالصيانة)

**Maintenance Prediction** = محاولة تقدير التغييرات المستقبلية قبل حدوثها لتخطيط الميزانية والتصميم المناسب.

**3 أهداف رئيسية للتنبؤ — Figure 9.13**

**3 عوامل للتنبؤ بطلبات التغيير:**

1. **عدد وتعقيد واجهات النظام** — واجهات أكثر = تغييرات مطلوبة أكثر.
2. **عدد المتطلبات المتغيّرة** — متطلبات السياسات المؤسسية أكثر تقلباً من متطلبات المجال الثابتة.
3. **عمليات الأعمال التي يخدمها النظام** — كلما اندمج مع عمليات أكثر = طلبات تغيير أكثر.

### 4 مقاييس لتقدير قابلية الصيانة (Process Metrics):

| المقياس | ما يدل عليه |
|---|---|
| **عدد طلبات Corrective Maintenance** | ارتفاعه = يُدخَل أخطاء أكثر مما يُصلَح = انخفاض القابلية |
| **متوسط وقت Impact Analysis** | ارتفاعه = مكونات متأثرة أكثر = انخفاض القابلية |
| **متوسط وقت تنفيذ طلب التغيير** | ارتفاعه = قابلية الصيانة تتراجع |
| **عدد طلبات التغيير المعلقة (Outstanding)** | ارتفاعه مع الزمن = انخفاض القابلية |

---

## Software Reengineering (إعادة الهندسة)

**Software Reengineering** = إعادة هيكلة وتوثيق الأنظمة القديمة لتسهيل فهمها وتعديلها — دون تغيير وظائفها.

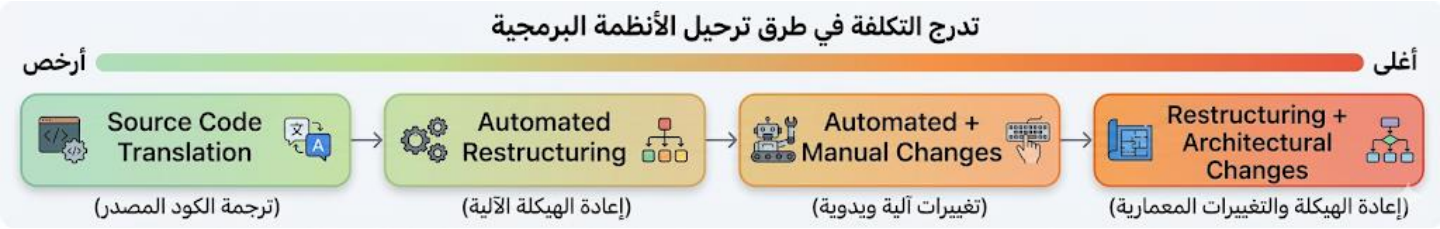
### مزايا Reengineering مقابل الاستبدال الكامل:

| المعيار | Reengineering | Replacement (إعادة بناء كامل) |
|---|---|---|
| **الخطر** | ✅ أقل خطراً | ❌ خطر عالٍ — قد يفشل المشروع |
| **التكلفة** | ✅ أقل تكلفة | ❌ أعلى تكلفة |
| **مثال حقيقي** | 12$ مليون | 50$ مليون (Ulrich 1990) |

### 5 أنشطة عملية Figure 9.14 — Reengineering

1. **Source Code Translation** ← تحويل لغة البرمجة (آلي)
2. **Reverse Engineering** ← استخراج الوثائق والبنية (آلي)
3. **Program Structure Improvement** ← تحسين هياكل التحكم (شبه آلي)
4. **Program Modularization** ← تجميع الأجزاء المترابطة (يدوي)
5. **Data Reengineering** ← تعديل قواعد البيانات والبيانات (مكلف جداً)

### طيف تكاليف Figure 9.15 — Reengineering




### حدود Reengineering — ماذا لا يستطيع أن يفعل؟

- لا يمكن تحويل نظام Functional إلى Object-Oriented.
- التغييرات المعمارية الكبيرة مكلفة جداً ولا يمكن أتمتتها.
- النظام المُعاد هندسته لن يكون بنفس جودة نظام حديث مبني من الصفر.

🔔 **Legacy System Wrapping** = تطوير خدمات Adaptor تُخفي الواجهات القديمة وتُقدّم واجهات حديثة منظّمة للمكونات الأخرى.

---

## 9.3.3 — Refactoring (إعادة الهيكلة)

**Refactoring** = تحسين برنامج لإبطاء تدهوره عبر التغييرات — تعديل بنيته وتبسيطه دون إضافة وظائف جديدة.

**الفكرة الأساسية:** Refactoring = "الصيانة الوقائية" — تمنع المشاكل قبل وقوعها.

### Refactoring vs Reengineering — الفرق الجوهري

| المعيار | Refactoring | Reengineering |
|---|---|---|
| **التوقيت** | مستمر خلال التطوير والتطوير | بعد سنوات من الصيانة |
| **الهدف** | منع التدهور مسبقاً | معالجة تدهور موجود |
| **الحجم** | تغييرات صغيرة متكررة | عملية ضخمة واحدة |
| **الأدوات** | دعم في IDE (Eclipse, etc.) | أدوات تحويل متخصصة |
| **في Agile؟** | ☑ جزء أصيل منه | ✕ ليس جزءاً من Agile |

### Bad Smells — علامات تستدعي Refactoring (Fowler et al. 1999)

| Bad Smell | الوصف | الحل |
|---|---|---|
| **Duplicate Code** | نفس الكود مكرر في أماكن متعددة | دمجه في method واحدة |
| **Long Methods** | method طويلة جداً | تقسيمها إلى methods أقصر |
| **Switch Statements** | Switch متكرر يعتمد على النوع | استخدام Polymorphism |
| **Data Clumping** | نفس مجموعة البيانات تتكرر في أماكن متعددة | تجميعها في Object/Class |
| **Speculative Generality** | كود مبني لاستخدام مستقبلي لم يحدث | حذفه ببساطة |

**أمثلة Refactoring Transformations (Fowler):**

- **Extract Method** → حذف التكرار وإنشاء method جديدة.
- **Consolidate Conditional Expression** → استبدال تسلسل tests بـ test واحد.
- **Pull Up Method** → استبدال methods متشابهة في Subclasses بـ method واحدة في Superclass.

⚠ **تحذير مهم:** إذا استلمت نظاماً بنيته متدهورة جداً، قد لا يكفي Refactoring للكود وحده — ستحتاج **Design Refactoring** = تحديد Design Patterns المناسبة واستبدال الكود القديم بكود يُنفّذها (Kerievsky 2004).

---

## Key Points — نقاط الفصل الكاملة

- هندسة البرمجيات = عملية حلزونية متكاملة (**Development + Evolution**).
- للبرامج المخصصة: تكاليف الصيانة تتجاوز تكاليف التطوير.
- **Legacy Systems** = أنظمة قديمة لا تزال ذات قيمة للأعمال.
- البقاء على Legacy أرخص وأقل خطراً من بناء نظام بديل في معظم الأحيان.
- **3 أنواع الصيانة:** Bug Fixing + Environmental Adaptation + New Requirements.
- **Reengineering** = إعادة هيكلة + توثيق للأنظمة الموجودة.
- **Refactoring** = تحسينات صغيرة مستمرة = صيانة وقائية.

---

## نقاط MCQ احفظها جيداً

- ◈ **58% من جهد الصيانة = Functionality Addition** — وليس Bug Fixing!
- ◈ Fault Repair | 19% = Environmental Adaptation = %24.
- ◈ **Requirements Errors** = الأغلى في الإصلاح (Fault Repair).
- ◈ **Lehman's Laws** — Belady 1970s & Lehman — البنية تتدهور مع كل تغيير.
- ◈ الصيانة أغلى من التطوير الأولي لنفس الوظيفة — 4 أسباب.
- ◈ الحل الجذري = دمج التطوير والصيانة في فريق واحد (مثال: **Amazon**).
- ◈ **COCOMO 2** = فهم الكود + كتابة كود جديد.
- ◈ **Reengineering** = بعد سنوات صيانة + لا يغيّر الوظيفة.
- ◈ **Reengineering** أرخص من Replacement — مثال: $50M12$ vs M.
- ◈ **5 أنشطة Reengineering:** Source Code Translation → Reverse Engineering → Structure Improvement → Modularization → Data Reengineering.
- ◈ **Refactoring** = صيانة وقائية مستمرة = جزء أصيل من Agile.
- ◈ **5 Bad Smells (Fowler):** Duplicate Code + Long Methods + Switch Statements + Data Clumping + Speculative Generality.
- ◈ الفرق الجوهري: **Refactoring** (مستمر + وقائي) ≠ **Reengineering** (لمرة + علاجي).
- ◈ **Legacy System Wrapping** = Adaptor Services لإخفاء الواجهات القديمة.